

BUILD-API.md

2. Principes communs

Version : V1

Statut : Référentiel officiel des principes de conception des API de Vitala

2.1 Objectif

Cette section définit les principes fondamentaux applicables à toutes les API de Vitala.

Elle constitue le cadre de référence pour la conception, le développement, les tests et la maintenance des interfaces de programmation de la plateforme.

Aucune API ne doit être développée en dehors de ces principes.

2.2 API First

Toutes les fonctionnalités exposées par Vitala doivent être conçues selon une approche **API First**.

Avant toute implémentation :

- les besoins métier sont identifiés ;
- le contrat d'API est défini ;
- les structures de données sont validées ;
- les règles de sécurité sont établies.

L'implémentation technique ne commence qu'après validation du contrat.

2.3 Domain Driven Design (DDD)

Les API sont organisées par domaines fonctionnels.

Chaque API appartient à un seul domaine métier.

Exemples :

- Identity API
- Flash API
- Mission API

- Social API
- Trust API
- Media API
- Notification API
- Settings API

Les domaines doivent rester faiblement couplés.

2.4 Responsabilité unique

Chaque endpoint doit avoir une responsabilité clairement définie.

Un endpoint ne doit effectuer qu'une seule action métier.

Les traitements complexes doivent être décomposés en plusieurs opérations cohérentes.

2.5 Cohérence

Toutes les API doivent respecter :

- les mêmes conventions de nommage ;
- les mêmes formats de réponse ;
- les mêmes codes HTTP ;
- les mêmes règles de sécurité ;
- les mêmes mécanismes de pagination ;
- les mêmes stratégies de filtrage.

L'expérience développeur doit rester homogène sur l'ensemble de la plateforme.

2.6 Stateless

Les API REST de Vitala sont **sans état (Stateless)**.

Chaque requête doit contenir toutes les informations nécessaires à son traitement.

Le serveur ne conserve aucun état de session entre deux requêtes.

2.7 Sécurité par défaut

Toute API est considérée comme protégée par défaut.

Les principes suivants s'appliquent :

- authentification obligatoire sauf exception documentée ;
 - autorisation contrôlée ;
 - validation stricte des permissions ;
 - application des politiques RLS ;
 - principe du moindre privilège.
-

2.8 Validation systématique

Toutes les données reçues doivent être validées avant traitement.

Les validations concernent notamment :

- les types ;
- les formats ;
- les longueurs ;
- les valeurs autorisées ;
- les règles métier.

Aucune donnée non valide ne doit être persistée.

2.9 Idempotence

Les opérations qui s'y prêtent doivent être idempotentes.

Une même requête exécutée plusieurs fois avec les mêmes paramètres doit produire un résultat identique lorsque cela est attendu.

Cette règle est particulièrement importante pour :

- les synchronisations ;
 - les paiements futurs ;
 - les traitements distribués ;
 - les Edge Functions.
-

2.10 Performance

Les API doivent être conçues pour minimiser :

- le temps de réponse ;
- le nombre de requêtes ;
- la quantité de données transférées ;
- les traitements inutiles.

Les optimisations ne doivent jamais compromettre la lisibilité ni la sécurité.

2.11 Observabilité

Chaque API doit être observable.

Les éléments suivants doivent pouvoir être suivis :

- temps d'exécution ;
- erreurs ;
- journaux ;
- métriques ;
- identifiant de corrélation (Correlation ID).

Ces informations facilitent le diagnostic et la supervision.

2.12 Compatibilité

Les évolutions des API doivent préserver la compatibilité autant que possible.

Les changements incompatibles doivent être :

- documentés ;
 - versionnés ;
 - annoncés avant leur mise en production.
-

2.13 Documentation obligatoire

Chaque endpoint doit être documenté.

La documentation doit préciser :

- son objectif ;
- les paramètres ;
- les réponses possibles ;
- les erreurs ;
- les permissions requises ;
- les exemples d'utilisation.

Une API non documentée est considérée comme incomplète.

2.14 Alignement architectural

Toutes les API doivent être cohérentes avec :

- VPS (Vision Produit & Système) ;
- Domain Model ;
- DATABASE-DICTIONARY ;
- DATABASE-RLS-MATRIX ;
- BUILD-ARCHITECTURE ;
- BUILD-SECURITY.

Les API ne doivent jamais contourner les règles définies par ces documents.

2.15 Conclusion

Les principes communs constituent le socle architectural de toutes les API de Vitala.

Ils garantissent des interfaces cohérentes, sécurisées, évolutives et faciles à maintenir, tout en offrant une expérience homogène aux développeurs, aux applications clientes et aux intelligences artificielles qui interagiront avec la plateforme.